

Задания к работе №2 по дисциплине “Алгоритмы и структуры данных”.

Все задания выполняются на языке программирования C++ (стандарт C++14 и выше).

Реализованные приложения не должны завершаться аварийно.

Во всех заданиях необходимо контролировать ситуации с невозможностью [пере]выделения памяти; во всех заданиях необходимо корректно освобождать всю выделенную динамическую память.

Все ошибки, связанные с операциями открытия системных ресурсов уровня ОС (файлы, средства синхронизации, etc.), должны быть обработаны; все открытые системные ресурсы должны быть возвращены ОС.

Во всех заданиях запрещено использование глобальных переменных.

Во всех заданиях сравнение (на предмет эквивалентности или отношения порядка) вещественных чисел на уровне функции должно использовать значение эпсилон, которое (предпочтительнее) является параметром этой функции, либо (на худой конец) является значением, определённым глобально.

Во всех заданиях при пользовательском вводе с консоли необходимо организовать приглашение пользователя ко вводу в соответствии с типом и бизнес-логикой запрашиваемых данных.

Для всех реализованных алгоритмов необходимо уметь объяснить их асимптотические сложности.

Во всех заданиях при реализации функций необходимо максимально ограничивать возможность модификации (если она не подразумевается) передаваемых в функцию параметров (используйте ключевое слово *const*).

Во всех заданиях все параметры функций и вводимые (с консоли, файла, командной строки) пользователем данные должны подвергаться валидации в соответствии с типом валидируемых данных, если не сказано обратное; валидация должна зависеть от типа данных и логики применения этих данных для выполнения целевой подзадачи. При передаче аргументов приложению через командную строку, их количество также должно валидироваться.

Во всех заданиях при реализации необходимо разделять контексты работы с данными (поиск, сортировка, добавление/удаление, модификация и т. п.) и отправка данных в поток вывода / выгрузка данных из потока ввода.

Во всех заданиях запрещено использование глобальных переменных (включая *errno*).

Во всех заданиях запрещено использование оператора безусловного перехода (*goto*).

Во всех заданиях запрещено пользоваться функциями, позволяющими завершить выполнение приложения из произвольной точки выполнения, вне контекста исполнения функции *main* в качестве точки входа (то есть вызванной не рекурсивно).

Во всех заданиях запрещено использование библиотеки стандартных шаблонов (STL).

Реализованные компоненты должны зависеть от абстракций, а не от конкретных реализаций абстракций. Для реализованных компонентов должны быть переопределены (либо перекрыты - при обосновании) следующие механизмы классов C++: конструктор копирования, деструктор, оператор присваивания.

1. Реализуйте приложение – интерпретатор операций над целочисленными массивами. Приложение оперирует целочисленными массивами произвольной длины с именами, являющимися непустыми строками длины, не превосходящей 16, которые могут содержать символы цифр ('0'..'9') и символы букв латинского алфавита ('A'..'Z', 'a'..'z'); имя массива не может начинаться с символа цифры. Система команд данного интерпретатора:

- Load *Abc*, *in.txt*; - загрузить в массив с именем *Abc* целые числа из файла *in.txt* (во входном файле могут произвольно присутствовать сепарирующие символы - пробелы, табуляции и переносы строк; также могут быть невалидные строковые репрезентации элементов массива);
- Save *AbcdE*, *out.txt*; - выгрузить элементы массива *AbcdE* в файл *out.txt*;
- Rand *Arr1*, *count*, *lb*, *rb*; - заполнить массив *Arr1* псевдослучайными элементами из отрезка [*lb*; *lr*] в количестве *count* штук.
- Concat *First*, *Second*; - сконкатенировать два массива *First* и *Second* результат сохранить в массив *First*;
- Free(*ToFree*); - очистить массив *ToFree* и сопоставить переменную *ToFree* с массивом из 0 элементов;
- Remove *RemoveFrom*, *2*, *7*; - удалить из массива *RemoveFrom* 7 элементов, начиная с элемента с индексом 2;
- Copy *A*, *4*, *10*, *b*; - скопировать из массива *A* элементы с 4 по 10 (оба конца включительно) и сохранить их в массив *b*, предварительно очистив его;
- Sort *A+*; - отсортировать элементы массива *A* по неубыванию;
- Sort *A-*; - отсортировать элементы массива *A* по невозрастанию;
- Shuffle *A*; - переставить элементы массива в псевдослучайном порядке;
- Stats *a*; - вывести в стандартный поток вывода статистическую информацию о массиве *A*: размер массива, максимальный и минимальный элемент (и их индексы), наиболее часто встречающийся элемент (если таковых несколько, вывести максимальный из них по значению), среднее значение элементов, максимальное из значений отклонений элементов от среднего значения;
- Print *a*, *3*; - вывести в стандартный поток вывода элемент массива *A* стоящий на позиции с номером 3;
- Print *a*, *4*, *16*; - вывести в стандартный поток вывода элементы массива *A*, начиная с 4 по 16 включительно оба конца;
- Print *a*, *all*; - вывести в стандартный поток вывод все элементы массива *A*.

Индексирование в массивах начинается с 0. Для реализации инструкции Sort используйте собственную реализацию алгоритма пирамидальной сортировки на основе структуры данных вида косая куча. Хранение массивов по строковым ключам организуйте на основе структуры данных вида префиксное дерево. Предоставьте текстовый файл с инструкциями для реализованного интерпретатора и продемонстрируйте его работу. Обработайте ошибки времени выполнения инструкций.

2. Реализуйте класс полинома от нескольких переменных. Имена переменных являются непустыми строками длины, не превосходящей 8, которые могут содержать символы цифр ('0'..'9') и символы строчных букв латинского алфавита ('a'..'z'); имя переменной не может начинаться с символа цифры. Мономы формируют полином на базе структуры данных вида односвязный список и должны быть организованы на базе структуры данных вида префиксное дерево; моном хранит значение вещественного коэффициента, а также для каждого имени переменной уровня монома хранится её степень (натуральное число). Для описанного класса полиномов реализовать операции сложения полиномов, вычитания полинома из полинома, умножения полиномов, умножения полинома на константу (вещественное число), взятия производной по заданной переменной (параметром является имя переменной). Продемонстрируйте работу реализованного функционала, обработав входной файл с выражениями, элементами которого являются полиномы. Формат файла продумайте самостоятельно. В результате обработки файла необходимо построить дерево каждого выражения с полиномами из файла, а также вывести результирующие полиномы в стандартный поток вывода.